Bluetooth Communication

Wireless Short-Range Communication

What is Bluetooth?

Think of Bluetooth as a **wireless conversation** between devices within a small room.

Real-world analogy:

- Like talking to someone sitting next to you in a coffee shop
- No wires needed, but limited range (~10 meters)
- Multiple people can join the conversation (up to one master with 8 slaves, but no slave to slave communication)

Technical definition:

- Short-range wireless communication protocol
- Uses 2.4 GHz ISM band (same as WiFi, but different protocols)
- Creates Personal Area Networks (PANs)

Why was Bluetooth Created?

Problem: Too many cables!

Bluetooth Low Energy (BLE): The Efficiency Expert

Classic Bluetooth vs BLE:

Aspect	Classic Bluetooth	Bluetooth Low Energy
Power	Like a desktop PC	Like a smartwatch
Range	~10 meters	~50 meters
Data Rate	Up to 3 Mbps	~1 Mbps
Use Case	Audio streaming, file transfer	IoT sensors, fitness trackers
Connection	Always connected	Connect when needed

Where Classic Bluetooth Is Still Used

Used when continuous, high-throughput streaming is needed:

- Wireless headphones (music)
- Car infotainment systems
- Hands-free calling
- Bluetooth speakers
- Serial modules (HC-05, SPP)

Why?

> Stable audio + sustained bandwidth

Where BLE Dominates

Used when low power + small data is required:

- Fitness trackers
- Smartwatches
- Medical sensors
- Beacons
- IoT nodes
- Smart locks

Why?

> Ultra-low energy consumption

Dual-Mode Bluetooth

- Some devices support both Classic and BLE (e.g., smartphones)
- Allows compatibility with a wider range of devices
- Classic for audio, BLE for fitness data
- It requires different stacks and profiles for each mode

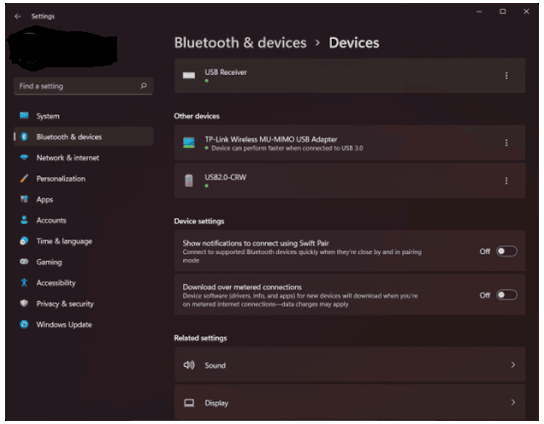
Summary: Key Technical Difference

Feature	Classic	BLE 4.2+
Audio Streaming	✅	❌ (until LE Audio)
Power Consumption	Higher	Very Low
Sensor Data	Possible	Optimized
IoT Use	Limited	Dominant

- Classic Bluetooth is **not obsolete**
  - Audio still relies on Classic (for now)
- BLE dominates IoT & wearables
  - BLE is perfect for IoT devices that need to run for months on a battery!

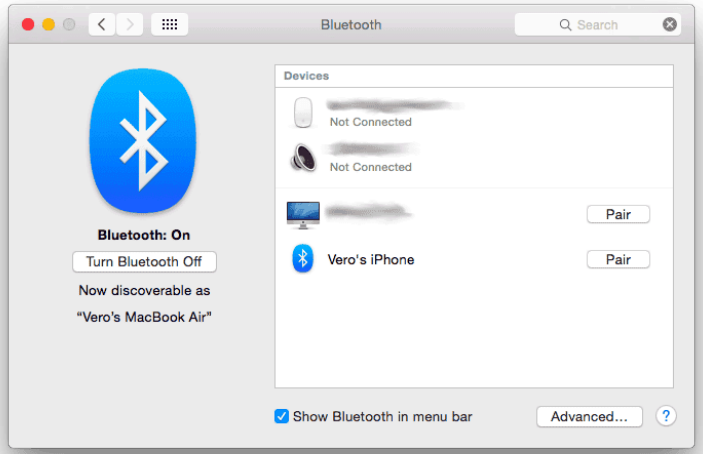
Bluetooth Connection Process: Like Making Friends

1. Discovery Phase (Finding friends)



2. Pairing Phase (Exchanging contact info)

- Devices exchange security keys
- Like exchanging phone numbers and passwords



3. Connection Phase (Start talking)

- Establish data channel
- Begin communication

Bluetooth Device Address (BD_ADDR)

Every Bluetooth device has a unique 48-bit address — similar to a MAC address in Wi-Fi.

```
Example:  2C:F0:A2:3B:1D:7E
          |-------||------|
        OUI (24b)  Device (24b)
```

- **OUI** (Organizationally Unique Identifier): assigned to the manufacturer by IEEE
- **Device ID**: assigned by the manufacturer per device

BD_ADDR Types

Type	Description	Example Use
Public Address	Globally unique, registered with IEEE	Classic BT, fixed BLE
Random Static	Random at power-on, stays until reset	BLE devices
Random Resolvable	Changes periodically (privacy protection)	BLE phones, wearables
Random Non-resolvable	Fully random, untrackable	BLE beacons

Why Random Addresses?

With a fixed Public Address, anyone nearby can **track your device**:

```
[9:00 AM]  AA:BB:CC:DD:EE:FF seen at coffee shop
[12:00 PM] AA:BB:CC:DD:EE:FF seen at office
[6:00 PM]  AA:BB:CC:DD:EE:FF seen at gym
```

BLE **Random Resolvable Private Address (RPA)** rotates periodically,
preventing tracking — but trusted devices can still resolve it using a shared key.

BD_ADDR in Practice

```
# Python: scan and print BD_ADDR of nearby devices
import bluetooth

devices = bluetooth.discover_devices(lookup_names=True)
for addr, name in devices:
    print(f"Address: {addr}  Name: {name}")
# Output:
# Address: 2C:F0:A2:3B:1D:7E  Name: My Headphones
```

```
# Linux: show your own Bluetooth address
hciconfig hci0
# BD Address: 2C:F0:A2:3B:1D:7E
```

UUID (Universally Unique Identifier) for BLE

- 128-bit identifier
- Format: `123e4567-e89b-12d3-a456-426614174000`
- Identifies **services, characteristics, or profiles**
- Used in BLE (GATT)

Key Differences between BD_ADDR and UUID:

Feature	BD_ADDR	UUID
Length	48-bit	128-bit
Identifies	Device	Service / Characteristic
Level	Hardware	Software
Used During	Discovery	After connection

Simple Analogy

- BD_ADDR → House address
- UUID → Room name inside the house

Device first, then services inside the device.

Bluetooth Security: Not Perfect, But Getting Better

Security Challenges:

```
Older Bluetooth (v2.0): PIN-based pairing
└── Problem: Easy to crack 4-digit PINs

Modern Bluetooth (v4.2+): Secure Simple Pairing
└── Uses elliptic curve cryptography
└── LE Secure Connections (introduced in v4.2)
└── Much harder to crack!
```

Question: Can Bluetooth be hacked when it uses Frequency Hopping?

Short Answer: YES

Frequency Hopping makes eavesdropping harder —
but **not impossible**.

> Frequency Hopping = Obfuscation
> Encryption = Real Security

Hopping hides *where* data is sent.
Encryption protects *what* data is sent.

Why?

- Hopping sequence can be **calculated** from connection data
- Attackers can **follow the hop pattern**
- Advanced radios can **capture wideband signals**
- Pairing phase may be attacked **before encryption**

Best Practices for Developers

- Always use latest Bluetooth version (at least v4.2+)
- Implement proper authentication
- Use encryption for sensitive data
- Make devices non-discoverable when not needed

Programming with Bluetooth: Tools & Libraries

Python

```
import bluetooth  # PyBluez library
```

Android (Java/Kotlin)

```
BluetoothAdapter adapter = BluetoothAdapter.getDefaultAdapter();
```

Cross-platform

- **Flutter** (cross-platform)
- **Web Bluetooth API** (Chrome)
- **React Native** (mobile apps)

Flutter supports Bluetooth through plugins like `flutter_blue`, allowing you to write once and run on both Android and iOS!

ESP32 (Using Arduino IDE)

```
#include "BluetoothSerial.h"
BluetoothSerial SerialBT;
```

- ESP32 has built-in Bluetooth/BLE support, making it a great choice for IoT projects!
- Using Arduino libraries simplifies development, but you can also use ESP-IDF for more control.

Warning! ESP32C6 does NOT support Bluetooth Classic!

ESP32-C6 is designed for:

- Ultra-low power IoT
- Matter / Thread / Zigbee
- Wi-Fi 6 + BLE

Classic Bluetooth requires:

- Higher power
- Audio stack
- Larger memory footprint

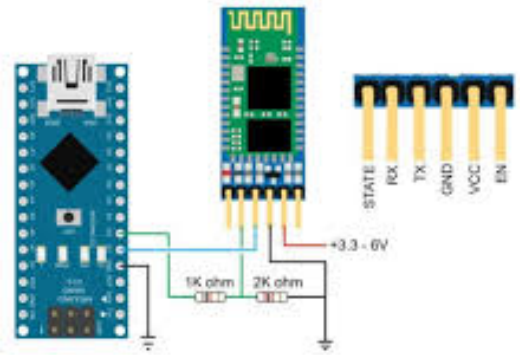
So Espressif removed it.

Bluetooth Module for Arduino

- HC-05/HC-06 (Classic Bluetooth)
- HM-10 (BLE)

HC05/HC06 (Classic Bluetooth)

HC05/HC06 Bluetooth module is a popular choice for microcontroller projects!



- We can use it to add Bluetooth capabilities to our projects without needing a full Bluetooth stack implementation.

SPP emulates a classic wired serial connection over Bluetooth.

It makes wireless communication behave like:

```
UART cable (TX/RX)
```

So your MCU code sees Bluetooth as just serial data.

Why SPP is Used in HC-05/HC-06?

Problem:

- Microcontrollers communicate via UART
- Phones/PCs use Bluetooth

Need a bridge between them.

Solution — SPP

SPP provides:

- Virtual COM port
- TX/RX byte streaming
- No packet formatting needed
- Simple terminal communication

Result:

> Arduino <-> Phone acts like Serial Monitor over air

Key Limitation

SPP works only with:

- Bluetooth Classic
- Devices that support serial profile

Not supported on:

- iOS BLE-only apps
- Modern BLE-exclusive devices

HM-10 (BLE)

HM-10 is a BLE module that supports GATT profiles, making it suitable for modern IoT applications.

What do I need to use HC05/06 or HM-10?

Your microcontroller communicates with the module via UART (HC-05/06) or a similar interface, sending AT commands to control Bluetooth behavior.

- We need to use AT commands to configure the module (e.g., set name, baud rate, pairing mode).

```
AT
→ OK
AT+VERSION?
→ +VERSION:2.0-20100601
AT+NAME?
→ +NAME:HC-05
```

Where is the Bluetooth stack in HC05/06 or HM-10?

- The Bluetooth stack is implemented in the module's firmware.
- The module handles all Bluetooth protocol details internally, so you don't have to implement the stack yourself.
- So, HC05/06 and HM-10 are not Bluetooth dongles that require a host stack; they are self-contained Bluetooth modules with their own stack.

Real-World Applications

In most cases for normal users:

Audio & Entertainment

- Wireless headphones/speakers
- Car audio systems
- Gaming controllers

nput Devices

- Wireless keyboards and mice
- Stylus for tablets

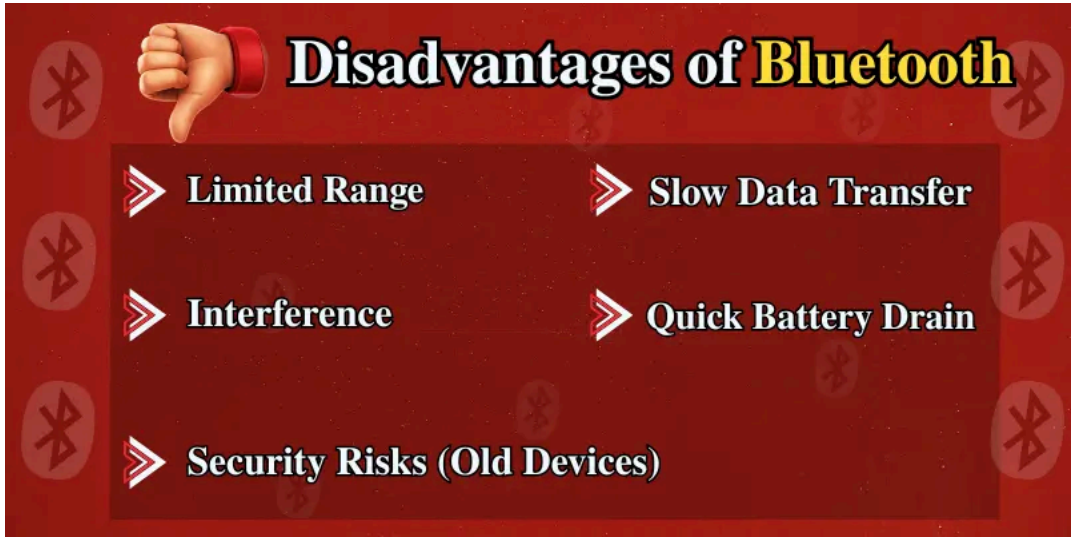
IoT & Sensors

- Fitness trackers (heart rate, steps)
- Smart home devices (thermostats, lights)
- Medical devices (glucose monitors)

Data Transfer

- File sharing between phones
- Backup and sync

Limitations of Bluetooth/BLE



- Not designed for long-range, high-bandwidth applications (like video streaming)
- For old devices, security can be weak (PIN-based pairing)

- It is not designed for high-quality audio streaming (though A2DP is good enough for most users).
- For Hi-Fi audio, WiFi-based solutions (like AirPlay) are often preferred.

Debugging Bluetooth: Common Issues

Connection Problems:

```
# Linux: Check Bluetooth status
bluetoothctl
> scan on
> devices
> connect XX:XX:XX:XX:XX:XX
```

Range Issues:

Bluetooth range depends on device class:

Bluetooth Class	Maximum Power	Operating Range
Class 1	100 mW (20 dBm)	100 meters (328 feet)
Class 2	2.5 mW (4 dBm)	10 meters (33 feet)
Class 3	1 mW (0 dBm)	1 meter (3 feet)

Interference:

- WiFi routers (same 2.4 GHz band)
- Microwave ovens
- Other Bluetooth devices

Solution: Move devices closer, reduce interference sources